

MERN Base Setup

Installation steps, login details and the git commands you will use every day.

This project is a ready-made starting point for a web application. It uses React on the front, Express on the back, and MongoDB for storage. Login, themes, menus and one sample feature are already built. You remove the sample and add your own features.

Front end - the part users see in the browser. **Back end** - the server that reads and writes the database. **Database** - where the information is stored permanently.

Project folder	mern-base-starter
Stack	React 19, Vite, Express 5, MongoDB, Mongoose
Language	Plain JavaScript and plain CSS
Tested on	Node 24, npm 11, local MongoDB - 31 of 31 tests passed

What is inside this guide

1. Before you start	2
2. Installation, step by step	2
3. Your login details	4
4. Commands you will use	4
5. Git - first time setup	6
6. Git - the daily loop	6
7. Git - checking your work	7
8. Git - branches	7
9. Git - undoing mistakes	8
10. Git - conflicts	8
11. What must never go to GitHub	9
12. When something goes wrong	9
13. One page memory sheet	10

1. Before you start

Install these three things first. Then check each one.

Tool	Version needed	Check with
Node.js	22.12 or newer	<code>node -v</code>
npm	comes with Node	<code>npm -v</code>
MongoDB	local, or free Atlas	<code>mongosh --eval "db.runCommand({ping:1})"</code>
Git	any recent version	<code>git --version</code>

If a command prints a version number, that tool is ready. If it says "command not found", install that tool first.

npm - the tool that downloads code libraries for your project. **Atlas** - MongoDB's free online database, if you do not want to install one.

2. Installation, step by step

Step 1. Unzip the file. Open a terminal inside the `mern-base-starter` folder.

Step 2. Create the settings files.

```
npm run setup
```

This makes two `.env` files. It also creates a random secret key for you. It never overwrites files you already have.

.env file - a private file holding passwords and settings. It never goes to GitHub.

Step 3. Open `backend/.env`. Set your database address.

```
# Local MongoDB
MONGO_URI=mongodb://127.0.0.1:27017/mern_base

# Or Atlas - paste the connection string from the Atlas website
MONGO_URI=mongodb+srv://user:password@cluster.mongodb.net/mern_base
```

Keep the database name at the end, such as `/mern_base`. The setting must be named `MONGO_URI` exactly.

Step 4. Download the libraries. This takes about half a minute.

```
npm install
```

Step 5. Optional. Create a demo account so you can sign in at once.

```
npm run seed
```

Step 6. Start the project. This runs the front end and back end together.

```
npm run dev
```

Step 7. Open `http://localhost:5173` in your browser.

Stop the servers with `Ctrl + C`. To run them in separate terminals, use `npm run dev:backend` and `npm run dev:frontend`.

3. Your login details

The application ships with no default account. Step 5 above creates this one.

Web address	http://localhost:5173
Email	student@example.com
Password	MernBase@2026

You can choose your own values instead:

```
SEED_EMAIL=you@example.com SEED_PASSWORD=YourStrongPass123 npm run seed
```

You can also skip the demo account. Click **Create an account** on the login page and register. Passwords must be at least 12 characters long.

This demo account is for your own computer only. Delete it before you put the project online. The seed command refuses to run in production for the same reason.

4. Commands you will use

Command	What it does
npm run setup	Creates the two .env files and a secret key. Run once.
npm install	Downloads the libraries. Run once, and after every pull.
npm run dev	Starts front end and back end together. Your daily command.
npm run seed	Creates the demo account and two sample rows.
npm test	Runs the quick tests. No database needed.
npm run test:api	Runs the full tests against a real test database.
npm run build	Builds the front end for the real server.
npm start	Runs the back end without the development tools.

Where things live

Folder	What is in it
frontend/src/pages/	One folder per screen the user sees.
frontend/src/components/	Buttons, fields and other reusable pieces.
frontend/src/services/	The functions that call the back end.
frontend/src/styles/theme.css	All the colours. Change your brand here.
backend/src/routes/	The web addresses your server answers.
backend/src/controllers/	What happens when an address is called.
backend/src/services/	The real work, including database calls.
backend/src/models/	The shape of each database record.
docs/	Guides, including how to add a feature.

5. Git - first time setup

Git saves the history of your code. GitHub keeps a copy online. Run every command from the project folder.

Repository - a project that git is tracking. Short form: repo. **Commit** - one saved point in your history. **Remote** - the online copy, usually on GitHub.

Tell git who you are. You do this once per computer.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Start tracking this project.

```
git init
git add .
git commit -m "First commit: MERN base setup"
```

Now create an empty repository on GitHub. Do not add a README there. Then connect your folder to it.

```
git remote add origin https://github.com/your-name/your-repo.git
git branch -M main
git push -u origin main
```

The `-u` links your branch to the online one. After this first push, plain `git push` is enough.

6. Git - the daily loop

These five commands are the cycle you repeat every day.

```
git pull                                # 1. Get the latest work from others
# ... write your code ...
git status                             # 2. See what you changed
git add .                             # 3. Choose the changes to save
git commit -m "Add login page"         # 4. Save them with a message
git push                              # 5. Send them to GitHub
```

Adding is called staging. Staging means choosing what goes into the next save. Committing saves that choice into your history.

Staging - a waiting area. Files sit here between `git add` and `git commit`.

Write messages that say what changed. "Fix login error message" is useful. "update" is not.

7. Git - checking your work

Command	What it shows
<code>git status</code>	Which files you changed.
<code>git diff</code>	What changed, line by line.
<code>git diff --staged</code>	What is already staged.
<code>git log --oneline --graph</code>	Your history, one line per commit.
<code>git log -p filename</code>	The history of one file.
<code>git show <commit-id></code>	Everything inside one commit.
<code>git remote -v</code>	Which GitHub repo you are linked to.

Adding and committing, in more detail

```
git add .                # everything that changed
git add filename         # one file only
git add folder/          # one folder
git reset filename       # unstage it, keep the edit

git commit -m "message"  # save what is staged
git commit -am "message" # stage tracked files and save together
git commit --amend -m "new" # fix the last message, before pushing
```

Getting other people's work

```
git pull                # download and merge in one step
git fetch               # download only, do not merge yet
git merge origin/main   # merge what you downloaded
```

Always run `git pull` before you start work. It prevents most conflicts.

8. Git - branches

A branch is a safe copy of your project. You build one feature there. Your main code stays untouched until the feature works.

```
git branch              # list your branches
git checkout -b feature-login # create one and switch to it
git checkout main       # switch back to main
git merge feature-login # bring the feature into main
git branch -d feature-login # delete it once merged
git push -u origin feature-login # put the branch on GitHub
```

9. Git - undoing mistakes

Command	What it does
<code>git restore filename</code>	Throws away your edits in one file.
<code>git restore --staged filename</code>	Unstages it, keeps the edit.
<code>git revert <commit-id></code>	Undoes a pushed commit safely.
<code>git reset --soft HEAD~1</code>	Undoes the last commit, keeps the changes.
<code>git reset --hard HEAD~1</code>	Undoes the last commit and deletes the changes.
<code>git stash</code>	Parks your changes for a moment.
<code>git stash pop</code>	Brings the parked changes back.

`--hard` destroys your work permanently. There is no undo. Use `git stash` instead when you are unsure.

10. Git - conflicts

A conflict happens when two people edit the same line. Git cannot guess which line is correct. It stops and asks you.

Fix it in four steps.

1. Run `git status`. It lists the files that conflict.
2. Open each file. Git has written marker lines into it.
3. Keep the correct code. Delete all three marker lines.
4. Save, then commit the result.

The markers look like this.

```
<<<<<< HEAD
your version of the line
=====
the other person's version
>>>>>> main
```

After you clean every file:

```
git add .
git commit -m "Resolve merge conflict"
git push
```

11. What must never go to GitHub

Two things must stay on your computer only.

- `.env` files - they hold your database password and secret key.
- `node_modules` - it is very large, and `npm install` rebuilds it anytime.

This project already blocks both in its `.gitignore` file. Check before your first push.

```
git status --short
```

If you see `.env` or `node_modules` in that list, stop. Fix `.gitignore` before you push anything.

If a `.env` file was committed by mistake:

```
git rm --cached backend/.env
git commit -m "Remove .env from git"
```

After that, treat the old secret as leaked. Delete `backend/.env` and run `npm run setup` again. That creates a fresh secret key.

12. When something goes wrong

What you see	What to do
MongoDB connection failed	Start your database. Check <code>MONGO_URI</code> in <code>backend/.env</code> . On Atlas, allow your current IP address.
Port 5000 is already in use	Another server is running. Stop it, or change <code>PORT</code> in <code>backend/.env</code> and <code>DEV_API_TARGET</code> in <code>frontend/.env</code> .
The message also names your shell	A <code>PORT</code> set by your terminal, Docker or PM2 wins over the <code>.env</code> file. Unset it, or change that value instead.
Cannot reach the API	The back end is not running. Look at the terminal for a red error line.
<code>JWT_SECRET</code> must be a random secret	You skipped step 2. Run <code>npm run setup</code> .
Blank white page	Open the browser console with F12. Read the first red line.
Changes do not appear	Hard refresh the page with <code>Ctrl + Shift + R</code> .

More detail is in `docs/TROUBLESHOOTING.md` inside the project.

13. One page memory sheet

Set up the project, once

```
npm run setup      # make .env files
# edit backend/.env, set MONGO_URI
npm install        # get libraries
npm run seed       # demo account
npm run dev        # start everything
```

Login

```
http://localhost:5173
student@example.com
MernBase@2026
```

Set up git, once

```
git config --global user.name "Your Name"
git config --global user.email "you@x.com"
git init
git add .
git commit -m "First commit"
git remote add origin <github-url>
git branch -M main
git push -u origin main
```

Every day

```
git pull
git status
git add .
git commit -m "What you changed"
git push
```

Look at things

```
git status      # what changed
git diff        # line by line
git log --oneline # short history
git remote -v   # which repo
```

Branches

```
git checkout -b feature-name
git checkout main
git merge feature-name
git branch -d feature-name
```

Undo

```
git restore file      # drop edits
git restore --staged file # unstage
git stash / git stash pop # park it
git revert <id>        # undo a push
```

Never commit

```
.env
node_modules
```

Keep this guide next to you for the first week. After that you will remember the daily five without looking.

- Your mentor